

UNIVERSIDAD DE NARIÑO
Especialización en Construcción de Software

Taller 2: Procesamiento y Persistencia de Datos con Spark

Fecha de entrega: Jueves 18 de Junio de 2026 a la 7:00 pm
al correo del profesor: oscar.o.ceballos@gmail.com

Información General

Aspecto	Detalle
Duración	2 horas
Modalidad	Práctico en parejas (2 estudiantes) o individual
Prerrequisitos	Guía 2 completada
Peso en nota	20% del total del curso
Entregable	Notebook taller_02_<apellido>_<nombre>.ipynb

Objetivo del Taller

Construir un pipeline de datos completo. El estudiante, como ingeniero de datos, debe:

1. **Extraer** datos de ventas desde un DataFrame creado manualmente
2. **Almacenar** en un Data Lake local (MinIO) en formato Delta Lake
3. **Transformar** con PySpark (limpieza, agregaciones, enriquecimiento)
4. **Cargar** resultados en PostgreSQL como Data Mart para reportes
5. **Validar** calidad de datos en cada etapa del pipeline

Actividades a Desarrollar

(10%) Actividad 1: Preparación del Ambiente

1. Crear SparkSession con appName "Taller2"
2. Configurar memoria: driver 1g, executor 1500m
3. Incluir el JAR de PostgreSQL JDBC:
(/usr/local/spark-3.5.8/jars/postgresql-42.7.3.jar)
4. Configurar S3A para MinIO: endpoint, credenciales, path-style access
5. Configurar extensiones Delta Lake
6. Imprimir: appName, master, sparkVersion, y confirmar que la sesión está activa

(15%) Actividad 2: Creación de Datos de Origen

1. Definir el esquema explícitamente con StructType (todos los tipos correctos: String, Date, Integer, Double).
2. Guardar el DataFrame como CSV en
~/spark_projects/data/ventas_datatech.csv (header=True).
3. Leer el CSV de vuelta y mostrar el esquema para confirmar tipos.
4. Mostrar los 10 registros.

(25%) Actividad 3: Data Lake Local - MinIO

1. Verificar que el bucket spark-lake existe en MinIO (usar boto3). Si no existe, crearlo.
2. Escribir el DataFrame df_ventas_raw en formato Delta Lake en:
s3a://spark-lake/datatech/ventas_raw. Usar mode("overwrite").
3. Leer de vuelta desde Delta y confirmar integridad (mismo número de registros y columnas).
4. Crear un DataFrame transformado df_ventas_clean con las siguientes transformaciones:
 - a. Agregar columna total_venta = cantidad * precio_unit
 - b. Agregar columna mes extrayendo el mes de fecha (como integer)
 - c. Agregar columna trimestre calculando: 1 si mes<=3, 2 si mes<=6, 3 si mes<=9, 4 si no
 - d. Agregar columna categoria_valor: "Premium" si total_venta > 1000, "Estándar" si > 200, "Básico" si no
 - e. Filtrar solo ventas del canal "Online"
5. Escribir df_ventas_clean en s3a://spark-lake/datatech/ventas_online en formato Delta, modo overwrite.
6. Verificar el historial de versiones de la tabla Delta ventas_online.
7. Realizar un MERGE (UPSERT): agregar 2 nuevas ventas (V011, V012) a ventas_raw. Si el id_venta ya existe, actualizar; si no, insertar.
8. Mostrar el historial actualizado de ventas_raw después del MERGE.

(30%) Actividad 4: Data Mart Relacional - PostgreSQL

1. Leer la tabla Delta ventas_online desde MinIO.
2. Crear una vista temporal v_ventas_online y ejecutar una consulta SQL que muestre:
 - a. categoria, trimestre, COUNT(*), SUM(total_venta), AVG(total_venta)
 - b. Ordenado por trimestre, categoría
3. Escribir el resultado de la consulta SQL en PostgreSQL en la tabla reporte_trimestral (modo overwrite).

4. Crear una segunda tabla en PostgreSQL: `top_clientes_online` con:
 - a. `id_cliente`, `ciudad`, `SUM(total_venta)`, `COUNT(*)`, `MAX(total_venta)`
 - b. Solo clientes con más de 1 compra online
 - c. Ordenado por total gastado descendente
 - d. Modo: `overwrite`
5. Leer ambas tablas de vuelta desde PostgreSQL y verificar que los conteos coinciden.
6. Agregar 1 registro nuevo a `reporte_trimestral` usando modo `append` (simulando carga incremental).

(30%) Actividad 5: Pipeline End-to-End

1. Función de Validación de Calidad

Crear una función `validar_calidad(df, nombre_tabla)` que verifique:

- Completitud: No hay valores nulos en columnas críticas (`id_venta`, `fecha`, `cantidad`, `precio_unit`).
- Unicidad: No hay `id_venta` duplicados.
- Rango: `cantidad > 0` y `precio_unit > 0`.
- Consistencia: `total_venta` (si existe) == `cantidad * precio_unit` (tolerancia 0.01).

La función debe imprimir un reporte y retornar (`True/False`, `count`) si pasa/falla todas las validaciones.

2. Ejecutar Validaciones en Cada Etapa

Ejecutar `validar_calidad` en:

- DataFrame original `df_ventas_raw` (después de crearlo)
- DataFrame en MinIO `ventas_raw` (después del MERGE)
- DataFrame transformado `ventas_online` (después de filtrar Online)
- DataFrame en PostgreSQL `reporte_trimestral` (después del `append`)

3. Reporte Final del Pipeline

Crear un DataFrame `df_reporte_pipeline` con las siguientes columnas:

- `etapa` (string): nombre de la etapa
- `registros` (int): número de registros
- `validacion` (string): "PASS" o "FAIL"
- `timestamp` (timestamp): momento de la validación

IMPORTANTE: Para crear el DataFrame de reporte, use una lista de tuplas de Python con `datetime.now()` y un schema explícito (`StructType`).

Escribir este reporte en PostgreSQL como tabla `log_pipeline` (modo `overwrite`).

Notebook: taller_2_su_nombre_y_apellido.ipynb

A continuación el contenido a modo de plantilla notebook que el estudiante debe completar:

1. Celda 1: Configuración Inicial

Python

```
# =====
# PUNTO 1: PREPARACIÓN DEL AMBIENTE
# =====

from pyspark.sql import SparkSession

spark = (SparkSession.builder
        .appName("TallerEvaluacionUnidad2")
        .master("local[*]")
        .config("spark.driver.memory", "1g")
        .config("spark.executor.memory", "1500m")
        # JDBC PostgreSQL
        .config("spark.jars", "/usr/local/spark-3.5.8/jars/postgresql-42.7.3.jar")
        # S3A MinIO
        .config("spark.hadoop.fs.s3a.endpoint", "http://localhost:9000")
        .config("spark.hadoop.fs.s3a.access.key", "minioadmin")
        .config("spark.hadoop.fs.s3a.secret.key", "minioadmin123")
        .config("spark.hadoop.fs.s3a.path.style.access", "true")
        .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem")
        .config("spark.hadoop.fs.s3a.connection.ssl.enabled", "false")
        # Delta Lake
        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog",
"org.apache.spark.sql.delta.catalog.DeltaCatalog")
        .getOrCreate())

print("=" * 60)
print("VERIFICACIÓN DE SPARKSESSION")
print("=" * 60)
print(f"App Name: {spark.sparkContext.appName}")
print(f"Master: {spark.sparkContext.master}")
print(f"Spark Version: {spark.version}")
print(f"Python Version: {spark.sparkContext.pythonVer}")
```

VERIFICACIÓN DE SPARKSESSION

App Name: TallerEvaluacionUnidad2
Master: local[*]
Spark Version: 3.5.8
Python Version: 3.12

2. Celda 2: Creación de Datos de Origen

id_venta	fecha	id_producto	producto	categoria	cantidad	precio_unit	id_cliente	ciudad	canal
V001	2024-01-15	P001	Laptop Pro	Tecnología	2	2500.00	C001	Bogotá	Online
V002	2024-01-16	P002	Mouse USB	Tecnología	5	25.50	C002	Medellín	Tienda
V003	2024-01-17	P003	Silla Ergonomica	Muebles	1	450.00	C003	Cali	Online
V004	2024-01-18	P001	Laptop Pro	Tecnología	1	2500.00	C001	Bogotá	Online
V005	2024-01-20	P004	Escritorio	Muebles	1	320.00	C004	Barranquilla	Tienda
V006	2024-02-05	P005	Monitor 27"	Tecnología	3	600.00	C002	Medellín	Online
V007	2024-02-10	P006	Lampara LED	Iluminación	4	35.00	C005	Cartagena	Online
V008	2024-02-12	P003	Silla Ergonomica	Muebles	2	450.00	C006	Bogotá	Tienda
V009	2024-03-01	P007	Tablet 10"	Tecnología	1	800.00	C001	Bogotá	Online
V010	2024-03-05	P008	Librero	Muebles	1	180.00	C007	Cali	Tienda

Python

```
# =====  
# PUNTO 2: CREACIÓN DE DATOS DE ORIGEN  
# =====  
  
from pyspark.sql.types import StructType, StructField, StringType, IntegerType,  
DoubleType, DateType  
from pyspark.sql.functions import col  
import os  
  
# TODO: Definir el esquema StructType con los 10 campos y tipos correctos  
# Pista: id_venta(String), fecha(DateType), id_producto(String), producto(String),  
#        categoria(String), cantidad(IntegerType), precio_unit(DoubleType),  
#        id_cliente(String), ciudad(String), canal(String)  
  
schema_ventas = StructType([  
    # TODO: Completar los 10 StructField  
])  
  
# TODO: Crear la lista de datos (mínimo 10 tuplas con los datos del enunciado)  
data_ventas = [  
    # TODO: Completar los 10 registros  
]  
  
# TODO: Crear DataFrame con spark.createDataFrame(data, schema)  
df_ventas_raw = None # Reemplazar  
  
print("=" * 60)
```

```

print("DATAFRAME DE VENTAS DATATECH SAS")
print("=" * 60)
# TODO: printSchema()
# TODO: show()

# TODO: Guardar como CSV en ~/projects/data/ventas_datatech.csv
csv_path = os.path.expanduser("~/spark_projects/data/ventas_datatech.csv")
# TODO: Escribir CSV con header=True, mode="overwrite"

print(f"\n✅ CSV guardado en: {csv_path}")

# TODO: Leer el CSV de vuelta con header=True e inferSchema=True
df_ventas_csv = None # Reemplazar

print("\n--- Esquema leído desde CSV ---")
# TODO: printSchema()
print("\n--- Datos leídos desde CSV ---")
# TODO: show()

```

```
=====
DATAFRAME DE VENTAS DATATECH SAS
=====
```

```
root
```

```

|-- id_venta: string (nullable = false)
|-- fecha: string (nullable = false)
|-- id_producto: string (nullable = false)
|-- producto: string (nullable = false)
|-- categoria: string (nullable = false)
|-- cantidad: integer (nullable = false)
|-- precio_unit: double (nullable = false)
|-- id_cliente: string (nullable = false)
|-- ciudad: string (nullable = false)
|-- canal: string (nullable = false)

```

id_venta	fecha	id_producto	producto	categoria	cantidad	precio_unit	id_cliente	ciudad	canal
V001	2024-01-15	P001	Laptop Pro	Tecnología	2	2500.0	C001	Bogotá	Online
V002	2024-01-16	P002	Mouse USB	Tecnología	5	25.5	C002	Medellín	Tienda
V003	2024-01-17	P003	Silla Ergonomica	Muebles	1	450.0	C003	Cali	Online
V004	2024-01-18	P001	Laptop Pro	Tecnología	1	2500.0	C001	Bogotá	Online
V005	2024-01-20	P004	Escritorio	Muebles	1	320.0	C004	Barranquilla	Tienda
V006	2024-02-05	P005	Monitor 27"	Tecnología	3	600.0	C002	Medellín	Online
V007	2024-02-10	P006	Lampara LED	Iluminación	4	35.0	C005	Cartagena	Online
V008	2024-02-12	P003	Silla Ergonomica	Muebles	2	450.0	C006	Bogotá	Tienda
V009	2024-03-01	P007	Tablet 10"	Tecnología	1	800.0	C001	Bogotá	Online
V010	2024-03-05	P008	Librero	Muebles	1	180.0	C007	Cali	Tienda

CSV guardado en: /home/test/projects/data/taller2/ventas_datatech.csv

--- Esquema leído desde CSV ---

root

```
|-- id_venta: string (nullable = true)
|-- fecha: date (nullable = true)
|-- id_producto: string (nullable = true)
|-- producto: string (nullable = true)
|-- categoria: string (nullable = true)
|-- cantidad: integer (nullable = true)
|-- precio_unit: double (nullable = true)
|-- id_cliente: string (nullable = true)
|-- ciudad: string (nullable = true)
|-- canal: string (nullable = true)
```

--- Datos leídos desde CSV ---

id_venta	fecha	id_producto	producto	categoria	cantidad	precio_unit	id_cliente	ciudad	canal
V006	2024-02-05	P005	Monitor 27"	Tecnología	3	600.0	C002	Medellín	Online
V007	2024-02-10	P006	Lampara LED	Iluminación	4	35.0	C005	Cartagena	Online
V008	2024-02-12	P003	Silla Ergonomica	Muebles	2	450.0	C006	Bogotá	Tienda
V009	2024-03-01	P007	Tablet 10"	Tecnología	1	800.0	C001	Bogotá	Online
V010	2024-03-05	P008	Librero	Muebles	1	180.0	C007	Cali	Tienda
V001	2024-01-15	P001	Laptop Pro	Tecnología	2	2500.0	C001	Bogotá	Online
V002	2024-01-16	P002	Mouse USB	Tecnología	5	25.5	C002	Medellín	Tienda
V003	2024-01-17	P003	Silla Ergonomica	Muebles	1	450.0	C003	Cali	Online
V004	2024-01-18	P001	Laptop Pro	Tecnología	1	2500.0	C001	Bogotá	Online
V005	2024-01-20	P004	Escritorio	Muebles	1	320.0	C004	Barranquilla	Tienda

3. Celda 3: Actividad 2 - Transformaciones

Python

```
# =====
# PUNTO 3: DATA LAKE LOCAL - MINIO
# =====

import boto3
from botocore.exceptions import ClientError
from pyspark.sql.functions import col, month, when, lit
from delta import DeltaTable

# 3.1 Verificar/crear bucket con boto3
print("=" * 60)
print("3.1 VERIFICACIÓN DE BUCKET MINIO")
print("=" * 60)

# TODO: Crear cliente boto3 apuntando a MinIO
# Pista: endpoint_url='http://localhost:9000', aws_access_key_id='minioadmin',
# etc.
s3 = None # Reemplazar

bucket_name = "spark-lake"
# TODO: Verificar si existe con head_bucket, si no, crearlo
```

```

# TODO: Imprimir resultado

# 3.2 Escribir datos crudos en Delta Lake sobre MinIO
print("\n" + "=" * 60)
print("3.2 ESCRITURA DELTA EN MINIO (ventas_raw)")
print("=" * 60)

delta_raw_path = "s3a://spark-lake/datatech/ventas_raw"
# TODO: Escribir df_ventas_raw en formato delta, modo overwrite

print(f"✅ Datos crudos escritos en: {delta_raw_path}")

# 3.3 Leer de vuelta y verificar integridad
# TODO: Leer desde delta_raw_path
df_raw_check = None # Reemplazar

# TODO: Verificar que count() y columns coinciden con df_ventas_raw
# TODO: Imprimir resultado de verificación

# 3.4 Transformaciones
print("\n" + "=" * 60)
print("3.4 TRANSFORMACIONES (ventas_online)")
print("=" * 60)

# TODO: Crear df_ventas_clean con las 5 transformaciones requeridas
# Pista: Usa withColumn() para cada nueva columna
# Pista: month(col("fecha")) extrae el mes
# Pista: when().when().otherwise() para trimestre y categoria_valor
# Pista: filter(col("canal") == "Online")

df_ventas_clean = None # Reemplazar

print("DataFrame transformado (solo Online):")
# TODO: printSchema() y show()

# 3.5 Escribir transformados en Delta
delta_online_path = "s3a://spark-lake/datatech/ventas_online"
# TODO: Escribir df_ventas_clean en formato delta, modo overwrite

print(f"\n Datos transformados escritos en: {delta_online_path}")

# 3.6 Historial de versiones
print("\n" + "=" * 60)
print("3.6 HISTORIAL DE VERSIONES (ventas_online)")
print("=" * 60)

```



```

# TODO: Crear DeltaTable.forPath() y mostrar history()

# 3.7 MERGE (UPSERT) en ventas_raw
print("\n" + "=" * 60)
print("3.7 MERGE (UPSERT) EN ventas_raw")
print("=" * 60)

# TODO: Crear DataFrame con 2-3 nuevas ventas (incluir una con id_venta existente
para probar update)
nuevas_ventas = [
    # TODO: Completar registros
]

# TODO: Crear DataFrame con el mismo schema_ventas
df_nuevas = None # Reemplazar

# TODO: Ejecutar MERGE: alias("target").merge(alias("source"),
condición).whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()

print("MERGE ejecutado exitosamente")

# 3.8 Historial actualizado
print("\n" + "=" * 60)
print("3.8 HISTORIAL ACTUALIZADO (ventas_raw)")
print("=" * 60)

# TODO: Mostrar historial actualizado y conteo de registros post-MERGE

```

3.1 VERIFICACIÓN DE BUCKET MINIO

Bucket 'spark-lake' existe

3.2 ESCRITURA DELTA EN MINIO (ventas_raw)

26/06/10 16:28:28 WARN MetricsConfig: Cannot locate configuration: tried hadoop-metrics2-s3a-file-system.properties,hadoop-metrics2.properties

Datos crudos escritos en: s3a://spark-lake/datatech/ventas_raw

26/06/10 16:28:37 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'sp

Integridad verificada: 10 registros, 10 columnas

3.4 TRANSFORMACIONES (ventas_online)

DataFrame transformado (solo Online):

```
root
|-- id_venta: string (nullable = false)
|-- fecha: string (nullable = false)
|-- id_producto: string (nullable = false)
|-- producto: string (nullable = false)
|-- categoria: string (nullable = false)
|-- cantidad: integer (nullable = false)
|-- precio_unit: double (nullable = false)
|-- id_cliente: string (nullable = false)
|-- ciudad: string (nullable = false)
|-- canal: string (nullable = false)
|-- total_venta: double (nullable = false)
|-- mes: integer (nullable = true)
|-- trimestre: integer (nullable = false)
|-- categoria_valor: string (nullable = false)
```

id_venta	fecha	id_producto	producto	categoria	cantidad	precio_unit	id_cliente	ciudad	canal	total_venta	mes	trimestre	categoria_valor
V001	2024-01-15	P001	Laptop Pro	Tecnología	2	2500.0	C001	Bogotá	Online	5000.0	1	1	Premium
V003	2024-01-17	P003	Silla Ergonomica	Muebles	1	450.0	C003	Cali	Online	450.0	1	1	Estándar
V004	2024-01-18	P001	Laptop Pro	Tecnología	1	2500.0	C001	Bogotá	Online	2500.0	1	1	Premium
V006	2024-02-05	P005	Monitor 27"	Tecnología	3	600.0	C002	Medellín	Online	1800.0	2	1	Premium
V007	2024-02-10	P006	Lampara LED	Iluminación	4	35.0	C005	Cartagena	Online	140.0	2	1	Básico
V009	2024-03-01	P007	Tablet 10"	Tecnología	1	800.0	C001	Bogotá	Online	800.0	3	1	Estándar

Datos transformados escritos en: s3a://spark-lake/datatech/ventas_online

3.6 HISTORIAL DE VERSIONES (ventas_online)

version	timestamp	userId	userName	operation	operationParameters	job	notebook	clusterId	readVersion	isolationLevel	isBlindAppend	operationMetrics
0	2026-06-10 16:28:49	NULL	NULL	WRITE	{mode -> Overwrite, partitionBy -> []}	NULL	NULL	NULL	NULL	Serializable	false	{numFiles -> 2, numOutputRows -> 6, numOutputBytes -> 8341}

3.7 MERGE (UPSERT) EN ventas_raw

MERGE ejecutado exitosamente

[illegible]

id_venta	fecha	id_producto	producto	categoria	cantidad	precio_unit	id_cliente	ciudad	canal
V001	2024-01-15	P001	Laptop Pro ACTUAL...	Tecnología	3	2500.0	C001	Bogotá	Online
V011	2024-03-10	P009	Auriculares BT	Tecnología	2	120.0	C008	Pereira	Online
V012	2024-03-12	P010	Mochila Laptop	Accesorios	1	85.0	C009	Manizales	Tienda

4. Celda 4: Actividad 3 - Consultas y Agregaciones

```
# =====
# PUNTO 4: DATA MART RELACIONAL - POSTGRESQL
# =====
```

```
# Configuración JDBC
jdbc_url = "jdbc:postgresql://localhost:5432/datamart"
connection_properties = {
    "user": "postgres",
    "password": "123abc",
    "driver": "org.postgresql.Driver"
}
```

```

# TODO: Leer desde delta_online_path en formato delta
df_online = None # Reemplazar

print(f"Registros leídos: {df_online.count()}")
# TODO: show(5)

# 4.2 Crear vista temporal y consulta SQL
print("\n" + "=" * 60)
print("4.2 CONSULTA SQL - REPORTE TRIMESTRAL")
print("=" * 60)

# TODO: Crear vista temporal v_ventas_online
# TODO: Ejecutar SQL con GROUP BY categoria, trimestre + agregaciones

df_reporte_sql = None # Reemplazar

print("Resultado de la consulta SQL:")
# TODO: show()

# 4.3 Escribir reporte trimestral en PostgreSQL
print("\n" + "=" * 60)
print("4.3 ESCRITURA EN POSTGRESQL (reporte_trimestral)")
print("=" * 60)

# TODO: Escribir df_reporte_sql en PostgreSQL, tabla reporte_trimestral, modo
overwrite

print("Tabla 'reporte_trimestral' creada en PostgreSQL")

# 4.4 Crear tabla top_clientes_online
print("\n" + "=" * 60)
print("4.4 TABLA TOP_CLIENTES_ONLINE")
print("=" * 60)

# TODO: groupBy id_cliente, ciudad. agg: sum(total_venta), count(*),
max(total_venta)
# TODO: filter num_compras > 1. orderBy total_gastado desc

df_top_clientes = None # Reemplazar

print("Top clientes online (más de 1 compra):")
# TODO: show()

# TODO: Escribir en PostgreSQL, tabla top_clientes_online, modo overwrite

```

```

print("Tabla 'top_clientes_online' creada en PostgreSQL")

# 4.5 Verificación de integridad
print("\n" + "=" * 60)
print("4.5 VERIFICACIÓN DE INTEGRIDAD")
print("=" * 60)

# TODO: Leer ambas tablas de PostgreSQL
# TODO: Comparar conteos con los DataFrames originales
# TODO: Imprimir resultados y assert

# 4.6 Append incremental
print("\n" + "=" * 60)
print("4.6 APPEND INCREMENTAL")
print("=" * 60)

# TODO: Crear 1 Row con datos de trimestre 2, categoría Tecnología
# TODO: Escribir en modo append a reporte_trimestral
# TODO: Leer de vuelta y mostrar conteo final

```

4.1 LECTURA DE DELTA DESDE MINIO

Registros leídos: 6

id_venta	fecha	id_producto	producto	categoria	cantidad	precio_unit	id_cliente	ciudad	canal	total_venta	mes	trimestre	categoria_valor
V001	2024-01-15	P001	Laptop Pro	Tecnología	2	2500.0	C001	Bogotá	Online	5000.0	1	1	Premium
V003	2024-01-17	P003	Silla Ergonomica	Muebles	1	450.0	C003	Cali	Online	450.0	1	1	Estándar
V004	2024-01-18	P001	Laptop Pro	Tecnología	1	2500.0	C001	Bogotá	Online	2500.0	1	1	Premium
V006	2024-02-05	P005	Monitor 27"	Tecnología	3	600.0	C002	Medellín	Online	1800.0	2	1	Premium
V007	2024-02-10	P006	Lampara LED	Iluminación	4	35.0	C005	Cartagena	Online	140.0	2	1	Básico

only showing top 5 rows

4.2 CONSULTA SQL - REPORTE TRIMESTRAL

Resultado de la consulta SQL:

categoria	trimestre	total_transacciones	ingresos_totales	ticket_promedio
Iluminación	1	1	140.0	140.0
Muebles	1	1	450.0	450.0
Tecnología	1	4	10100.0	2525.0

4.3 ESCRITURA EN POSTGRESQL (reporte_trimestral)

Tabla 'reporte_trimestral' creada en PostgreSQL

4.4 TABLA TOP_CLIENTES_ONLINE

Top clientes online (más de 1 compra):

id_cliente	ciudad	total_gastado	num_compras	compra_maxima
C001	Bogotá	8300.0	3	5000.0

Tabla 'top_clientes_online' creada en PostgreSQL

4.5 VERIFICACIÓN DE INTEGRIDAD

Registros en reporte_trimestral (PostgreSQL): 3
Registros en top_clientes_online (PostgreSQL): 1
Integridad verificada en ambas tablas

4.6 APPEND INCREMENTAL

Registros después del append: 4

categoria	trimestre	total_transacciones	ingresos_totales	ticket_promedio
Iluminación	1	1	140.0	140.0
Muebles	1	1	450.0	450.0
Tecnología	1	4	10100.0	2525.0
Tecnología	2	5	5500.0	1100.0

Append incremental completado

5. Celda 5: Actividad 4 - Modificación y Persistencia

Python

```
# =====  
# PUNTO 5: PIPELINE END-TO-END Y VALIDACIÓN DE CALIDAD  
# =====  
  
from pyspark.sql.functions import col, isnan, when, count, abs as spark_abs, lit  
from pyspark.sql.types import TimestampType, StructType, StructField, StringType,  
IntegerType  
from pyspark.sql import Row  
from datetime import datetime  
  
# 5.1 Función de validación de calidad  
def validar_calidad(df, nombre_tabla):  
    """
```

```

Valida la calidad de datos de un DataFrame.
Retorna (True/False, count) si pasa/falla todas las validaciones.
"""

print(f"\n{'='*60}")
print(f"VALIDACIÓN DE CALIDAD: {nombre_tabla}")
print(f"{'='*60}")

# TODO: Implementar las 4 validaciones requeridas
# Pista 1: Completitud - count(when(col().isNull() | isnan(col()), col()))
# Pista 2: Unicidad - groupBy("id_venta").count().filter(col("count") > 1)
# Pista 3: Rango - filter(col("cantidad") <= 0)
# Pista 4: Consistencia - abs(col("total_venta") - (col("cantidad") *
col("precio_unit"))) > 0.01

# TODO: Imprimir reporte con PASS / FAIL por cada validación
# TODO: Retornar (all_pass, total_registros)

pass # Eliminar esta línea

# 5.2 Ejecutar validaciones en cada etapa
print("=" * 60)
print("5.2 VALIDACIONES EN CADA ETAPA DEL PIPELINE")
print("=" * 60)

log_pipeline = []

# TODO: Etapa 1 - Validar df_ventas_raw
# ok1, count1 = validar_calidad(..., "...")
# log_pipeline.append(..., count1, "PASS" if ok1 else "FAIL")

# TODO: Etapa 2 - Validar ventas_raw post-MERGE desde MinIO
# ok2, count2 = validar_calidad(..., "...")

# TODO: Etapa 3 - Validar ventas_online desde MinIO
# ok3, count3 = validar_calidad(..., "...")

# TODO: Etapa 4 - Validar reporte_trimestral desde PostgreSQL
# Nota: Para esta tabla, id_venta no existe. Imprime conteo y marca como PASS.

# 5.3 Reporte final del pipeline
print("\n" + "=" * 60)
print("5.3 REPORTE FINAL DEL PIPELINE")
print("=" * 60)

# CORRECCIÓN CLAVE: Crear datos como lista de tuplas Python con datetime.now()

```

```

# NO usar Row() con current_timestamp() de Spark

# TODO: Crear lista de tuplas (etapa, registros, validacion, datetime.now())
data_for_reporte = []
# for etapa, registros, validacion in log_pipeline:
#     data_for_reporte.append((etapa, registros, validacion, datetime.now()))

# TODO: Crear schema explícito con StructType para el reporte
# schema_reporte = StructType([...])

# TODO: Crear DataFrame con spark.createDataFrame(data_for_reporte,
# schema=schema_reporte)
df_reporte_pipeline = None # Reemplazar

print("Reporte del pipeline:")
# TODO: show(truncate=False)

# TODO: Escribir df_reporte_pipeline en PostgreSQL, tabla log_pipeline, modo
# overwrite

print("\nTabla 'log_pipeline' creada en PostgreSQL")

# TODO: Leer de vuelta y mostrar verificación

```


5.2 VALIDACIONES EN CADA ETAPA DEL PIPELINE

VALIDACIÓN DE CALIDAD: df_ventas_raw (origen)

Total registros: 10

Compleitud	PASS - Nulos encontrados: 0
Unicidad	PASS - Duplicados: 0
Rango (cantidad)	PASS - Inválidas: 0
Rango (precio)	PASS - Inválidos: 0
Consistencia	PASS - Columnas no aplica

Resultado final: TODAS LAS VALIDACIONES PASARON

VALIDACIÓN DE CALIDAD: ventas_raw (MinIO post-MERGE)

Total registros: 12

Compleitud	PASS - Nulos encontrados: 0
Unicidad	PASS - Duplicados: 0
Rango (cantidad)	PASS - Inválidas: 0
Rango (precio)	PASS - Inválidos: 0
Consistencia	PASS - Columnas no aplica

Resultado final: TODAS LAS VALIDACIONES PASARON

VALIDACIÓN DE CALIDAD: ventas_online (transformados)

Total registros: 6

Compleitud	PASS - Nulos encontrados: 0
Unicidad	PASS - Duplicados: 0
Rango (cantidad)	PASS - Inválidas: 0
Rango (precio)	PASS - Inválidos: 0
Consistencia	PASS - Inconsistencias: 0

Resultado final: TODAS LAS VALIDACIONES PASARON

VALIDACIÓN DE CALIDAD: reporte_trimestral (PostgreSQL)

Total registros: 4

PASS - Tabla agregada, validación estructural omitida

5.3 REPORTE FINAL DEL PIPELINE

Reporte del pipeline:

etapa	registros	validacion	timestamp
Origen CSV	10	PASS	2026-06-10 16:45:14.506763
MinIO post-MERGE	12	PASS	2026-06-10 16:45:14.50677
Transformados Online	6	PASS	2026-06-10 16:45:14.506771
PostgreSQL reporte	4	PASS	2026-06-10 16:45:14.506772

Tabla 'log_pipeline' creada en PostgreSQL

Verificación desde PostgreSQL:

etapa	registros	validacion	timestamp
Origen CSV	10	PASS	2026-06-10 16:45:14.506763
MinIO post-MERGE	12	PASS	2026-06-10 16:45:14.50677
Transformados Online	6	PASS	2026-06-10 16:45:14.506771
PostgreSQL reporte	4	PASS	2026-06-10 16:45:14.506772

Evaluación

La sumatoria total de los puntos es igual a 100, lo que es equivalente a una nota de 5.0 en el parcial. En todos los casos la sustentación será pilar fundamental de la nota final asignada. Cada estudiante, después de la entrega del taller tendrá asignado un factor de sustentación (un número real entre 0 y 1), correspondiente al grado de calidad de la sustentación. La nota definitiva será la nota del proyecto multiplicada por el factor de sustentación.

Por ejemplo, si la nota en el taller es 5.0 y el valor del factor de sustentación es 1, la nota final será 5.0. Pero si el factor de sustentación es 0.5, la nota final será 2.5. La no presentación de la sustentación tendrá como resultado una asignación de un factor de 0. Lo que no sea debidamente sustentado no vale así funcione muy bien.